AI LAB # 2

ROLL NO : 24F-3054

Question # 1 :

[5]:
```
class StorageMax:
    def __init__(self, length, width, height):
        self.length =length
        self.width = width
        self.height = height

    def Calculate_area(self):
        return (self.length*self.width)

    def Calculate_volume(self):
        return (self.length*self.width*self.height)

    def display(self):
        print(f"Area : {self.Calculate_area()} and Volume : {self.Calculate_volume()}")
        print(f"Storage Allocation Completed ......")
values = (
    int(input("Enter length : ")),
    int(input("Enter Width : ")),
    int(input("Enter height : "))
)


a = StorageMax(values[0], values[1], values[2])

a.display()
```

```
Enter length :  1
Enter Width :  2
Enter height :  3
Area : 2 and Volume : 6
Storage Allocation Completed ......
```

Question # 2:

[8]:
```python
class FitTrack:
    def __init__(self):
        self.Cardio = set()
        self.Strength = set()
        self.Flexibility = set()
        self.Endurance = set()

    def addExercise(self, category, exercise):
        category.add(exercise)

    def removeExercise(self, category, exercise):
        category.remove(exercise)

    def checkExercise(self, category, exercise):
        print(f"Exercise Exists : {exercise in category}")

    def exerciseUnion(self, set1, set2):
        return set1.union(set2)

    def exerciseIntersection(self, set1, set2):
        return set1.intersection(set2)

    def exerciseDifference(self, set1, set2):
        return set1.difference(set2)
fit = FitTrack()

fit.addExercise(fit.Cardio, "Running")
fit.addExercise(fit.Cardio, "Cycling")
fit.addExercise(fit.Strength, "Bench Press")
fit.addExercise(fit.Strength, "Squats")

fit.checkExercise(fit.Cardio, "Running")
fit.checkExercise(fit.Cardio, "Swimming")

print(fit.exerciseUnion(fit.Cardio, fit.Strength))
print(fit.exerciseIntersection(fit.Cardio, fit.Strength))
print(fit.exerciseDifference(fit.Cardio, fit.Strength))
```

```
Exercise Exists : True
Exercise Exists : False
```

```
Exercise Exists : True
Exercise Exists : False
{'Squats', 'Cycling', 'Running', 'Bench Press'}
set()
{'Cycling', 'Running'}
```

Question # 3 :

```
{'Cycling', 'Running'}
```

[13]:
```
Hazards = {
    "Exposed electrical wiring" : 5,
    "Gas leak smell" : 5,
    "Water leakage or mold growth" : 4,
    "Loose stair railings" : 3,
    "Faulty smoke detectors" : 4,
    "Clogged or blocked exits" : 3,
    "Slippery floors without mats" : 3,
    "Windows or doors that don't lock properly" : 4
}

temp = 0
for hazard, severity in Hazards.items():
    print(f"{hazard} , is you notice this hazard (Yes = 1 ,No = 2) ")
    choice = int(input("Enter your Choice : "))
    if(choice == 1):
        temp = temp + severity
print(f"Hazards Score : {temp}")
if(temp > 15 ):
    print("Based on the hazards reported, your home may require safety improvements. It is recommended to address
else:
    print("Based on the hazards reported, your home does not seem to have critical safety risks. However, regular
```

```
Exposed electrical wiring , is you notice this hazard (Yes = 1 ,No = 2)
Enter your Choice :  1
```

```
Exposed electrical wiring , is you notice this hazard (Yes = 1 ,No = 2)
Enter your Choice :  1
Gas leak smell , is you notice this hazard (Yes = 1 ,No = 2)
Enter your Choice :  1
Water leakage or mold growth , is you notice this hazard (Yes = 1 ,No = 2)
Enter your Choice :  1
Loose stair railings , is you notice this hazard (Yes = 1 ,No = 2)
Enter your Choice :  1
Faulty smoke detectors , is you notice this hazard (Yes = 1 ,No = 2)
Enter your Choice :  1
Clogged or blocked exits , is you notice this hazard (Yes = 1 ,No = 2)
Enter your Choice :  1
Slippery floors without mats , is you notice this hazard (Yes = 1 ,No = 2)
```

[ ]:

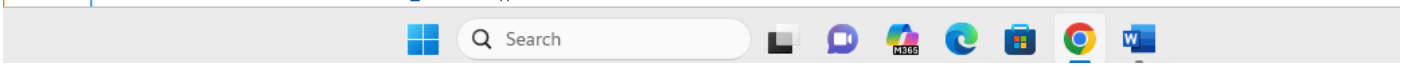
```
Clogged or blocked exits , is you notice this hazard (Yes = 1 ,No = 2)
Enter your Choice :  1
Slippery floors without mats , is you notice this hazard (Yes = 1 ,No = 2)
Enter your Choice :  1
Windows or doors that don't lock properly , is you notice this hazard (Yes = 1 ,No = 2)
Enter your Choice :  1
Hazards Score : 31
Based on the hazards reported, your home may require safety improvements. It is recommended to address these issues to prevent accidents.
```

[ ]:

Question # 4 :

•[15]:
```python
class Product:
    def __init__(self, name, price, stock):
        self.name = name
        self.price = price
        self.stock = stock

    def is_available(self, quantity):
        if self.stock >= quantity:
            return True
        else:
            return False

    def reduce_stock(self, quantity):
        if self.is_available(quantity):
            self.stock -= quantity
        else:
            print("Not enough Stocks available")

    def display(self):
        print(f"{self.name} (Rs. {self.price:.2f}, Stock: {self.stock})")


class OrderItem:
    def __init__(self, product, quantity):
        self.product = product
        self.quantity = quantity

    def get_total_price(self):
        return self.product.price * self.quantity

    def display(self):
        print(
            f"{self.product.name} x {self.quantity} = "
            f"Rs. {self.get_total_price():.2f}"
        )


class Customer:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    def make_payment(self, amount):
        if self.balance >= amount:
```

```
    def make_payment(self, amount):
        if self.balance >= amount:
            self.balance -= amount
            return True
        return False

    def display(self):
        print(f"{self.name} (Balance: Rs. {self.balance:.2f})")


class Order:
    TAX_RATE = 0.05
    DISCOUNT_THRESHOLD = 5000
    DISCOUNT_RATE = 0.10

    def __init__(self, customer):
        self.customer = customer
        self.items = []

    def add_item(self, product, quantity):
        if product.is_available(quantity):
            self.items.append(OrderItem(product, quantity))
            return True
        else:
            print(
                f"Cannot add {product.name}: "
                f"only {product.stock} stocks available."
            )
            return False

    def calculate_subtotal(self):
        total = 0

        for item in self.items:
            total += item.get_total_price()

        return total

    def apply_discount(self, subtotal):
        if subtotal >= self.DISCOUNT_THRESHOLD:
            return subtotal * self.DISCOUNT_RATE
        else:
            return 0

    def calculate_total(self):
        subtotal = self.calculate_subtotal()
```



```
    def calculate_total(self):
        subtotal = self.calculate_subtotal()

        discount = self.apply_discount(subtotal)

        taxable_amount = subtotal - discount

        tax = taxable_amount * self.TAX_RATE

        total = taxable_amount + tax

        return subtotal, discount, tax, total

    def process_order(self):

        for item in self.items:
            if not item.product.is_available(item.quantity):
                print(
                    f"Order failed: {item.product.name} "
                    f"does not have enough stock."
                )
                return False

        subtotal, discount, tax, total = self.calculate_total()

        print("Order Summary ")

        for item in self.items:
            item.display()

        print(f"Subtotal: Rs. {subtotal:.2f}")
        print(f"Discount: -Rs. {discount:.2f}")
        print(f"Tax (5%): +Rs. {tax:.2f}")
        print(f"Total Due: Rs. {total:.2f}")


        if self.customer.make_payment(total):

            for item in self.items:
                item.product.reduce_stock(item.quantity)

            print(f"Payment successful! Remaining balance: Rs. {self.customer.balance:.2f}")

            return True
```

Code JupyterLab Python 3.13 (XPython)

```
laptop = Product("Laptop", 80000, 5)
mouse = Product("Wireless Mouse", 1500, 20)
keyboard = Product("Mechanical Keyboard", 4500, 10)

dawood = Customer("Muhammad Dawood", 100000)

order = Order(dawood)

order.add_item(laptop, 1)
order.add_item(mouse, 2)
order.add_item(keyboard, 1)

order.process_order()

print("\n--- Updated Stock ---")

laptop.display()
mouse.display()
keyboard.display()

print("\n--- Customer Information ---")
dawood.display()
```

```
--- Order Summary ---
Laptop x 1 = Rs. 80000.00
Wireless Mouse x 2 = Rs. 3000.00
Mechanical Keyboard x 1 = Rs. 4500.00
Subtotal: Rs. 87500.00
Discount: -Rs. 8750.00
Tax (5%): +Rs. 3937.50
Total Due: Rs. 82687.50
Payment successful! Remaining balance: Rs. 17312.50

--- Updated Stock ---
Laptop (Rs. 80000.00, Stock: 4)
Wireless Mouse (Rs. 1500.00, Stock: 18)
Mechanical Keyboard (Rs. 4500.00, Stock: 9)

--- Customer Information ---
Muhammad Dawood (Balance: Rs. 17312.50)
```

[ ]: